1. DOCUMENTO ARQUITECTÓNICO

Descripción del problema:

- ¿Qué problema resuelve?

En muchas entidades bancarias, la asignación de turnos para la atención de los usuarios puede generar desorden y tiempos de espera poco claros, especialmente cuando no existe un sistema que permita organizar adecuadamente las prioridades de atención. En estos casos, los usuarios deben esperar sin conocer con precisión cuánto tiempo tardarán en ser atendidos, lo que puede generar inconformidad y dificultades en la organización del servicio.

Para dar solución a esta situación, se propone el desarrollo de un sistema digital de gestión de turnos que permita organizar la atención de los usuarios de manera sencilla. En el sistema, el usuario podrá solicitar un turno ingresando únicamente su número de identificación y su número de teléfono, sin necesidad de crear una cuenta o iniciar sesión. Posteriormente, el usuario seleccionará el tipo de atención que requiere, que puede ser atención general o atención preferencial. A partir de esta información, el sistema generará un turno alfanumérico y organizará la lista de espera teniendo en cuenta la prioridad seleccionada.

- ¿Por qué necesita ser distribuido?

El sistema se plantea como una solución distribuida debido a que sus diferentes funciones se gestionan mediante servicios independientes que trabajan de forma coordinada. En lugar de tener una sola aplicación encargada de todas las tareas, el sistema divide sus responsabilidades en varios servicios que cumplen funciones específicas, como el registro de usuarios, la gestión de turnos y el envío de notificaciones.

Esta separación permite que cada servicio funcione de manera independiente, lo que facilita el mantenimiento del sistema y permite realizar mejoras o cambios en un servicio sin afectar directamente a los demás. Además, este enfoque mejora la organización del sistema y permite que cada componente pueda escalar o modificarse según las necesidades del sistema. De esta forma, el uso de una arquitectura distribuida contribuye a crear una solución más flexible, modular y fácil de administrar.

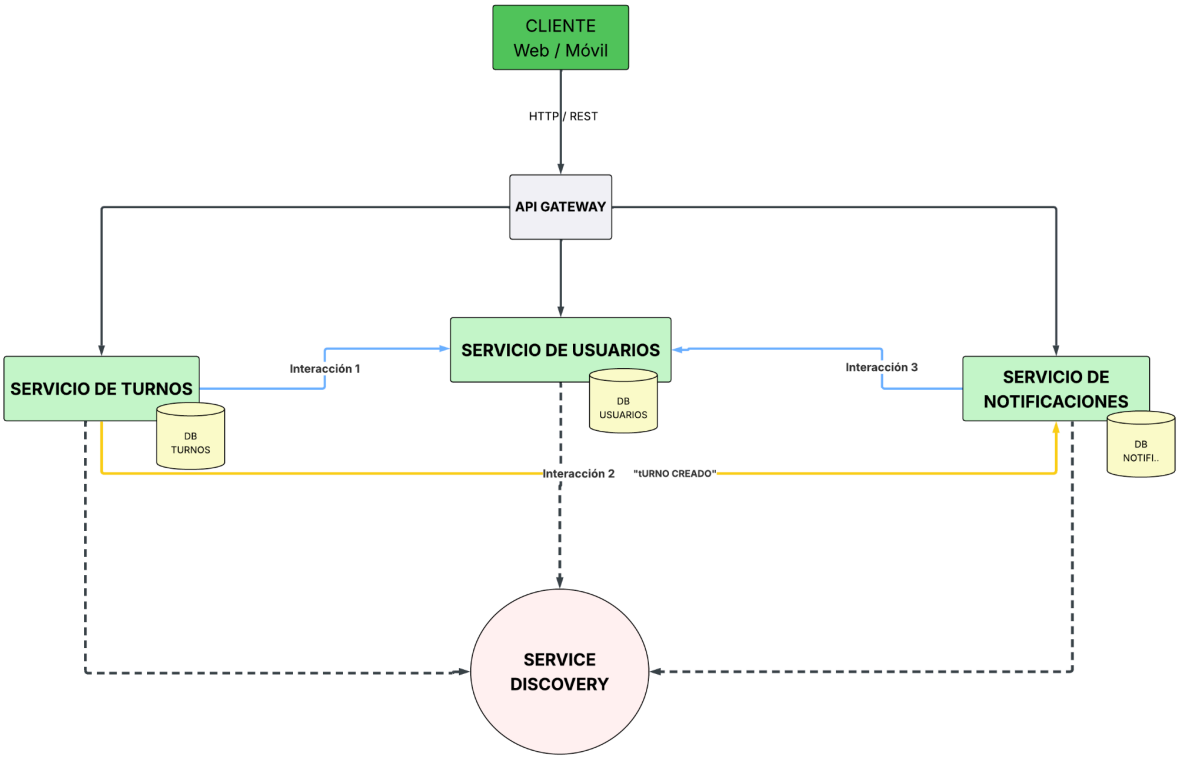
Arquitectura propuesta:

- Microservicios:

La arquitectura propuesta para el sistema se basa en un modelo de microservicios bajo un esquema cliente-servidor. En este modelo, el usuario interactúa con el sistema a través de una aplicación o interfaz que actúa como cliente, desde donde se solicita el turno ingresando la información básica requerida. Esta solicitud es enviada al servidor, donde diferentes servicios se encargan de procesar la información y ejecutar las funciones necesarias.

Dentro del sistema se implementan tres microservicios principales que trabajan de manera independiente pero coordinada. Cada servicio se encarga de una función específica del sistema, lo que permite mantener una arquitectura modular y organizada. Los servicios se comunican mediante peticiones HTTP utilizando una API REST, lo que permite intercambiar información entre ellos de forma sencilla. Además, cada servicio se ejecuta dentro de un contenedor Docker, lo que facilita su despliegue y ejecución dentro del entorno del sistema.

- Diagrama:



Identificación de servicios

- Servicio de usuarios:

El servicio de usuarios se encarga de gestionar la información básica de los usuarios que solicitan un turno dentro del sistema. Su función principal es registrar y consultar los datos del usuario, específicamente el número de identificación y el número de teléfono. Este servicio permite almacenar y recuperar la información necesaria para identificar al usuario dentro del sistema y facilitar la generación de turnos.

- Servicio de gestión de turnos:

El servicio de gestión de turnos es el encargado de generar y organizar los turnos solicitados por los usuarios. Cuando un usuario solicita un turno, este servicio crea un identificador alfanumérico que representa el turno asignado y lo organiza dentro de la lista de espera. El sistema tiene en cuenta la prioridad de atención seleccionada por el usuario, diferenciando entre atención general y atención preferencial, lo que permite organizar la atención de manera más ordenada.

- Servicio de notificaciones:

El servicio de notificaciones se encarga de enviar información al usuario relacionada con el turno que ha sido asignado. Este servicio permite comunicar al usuario el número de turno generado y el tiempo estimado de espera para ser atendido. Su función es mantener informado al usuario sobre el estado de su turno y mejorar la experiencia durante el proceso de atención.

Arquitecturas a usar:

- Backend:

Para el desarrollo del backend del sistema se utilizará Python junto con el framework Flask, el cual permite crear servicios web de manera sencilla y exponer endpoints que pueden ser utilizados por los diferentes componentes del sistema.

- Base de datos:

Para el almacenamiento de la información del sistema se utilizará MySQL, una base de datos relacional que permite gestionar de manera eficiente los datos relacionados con los usuarios y los turnos generados por el sistema.

- Comunicación:

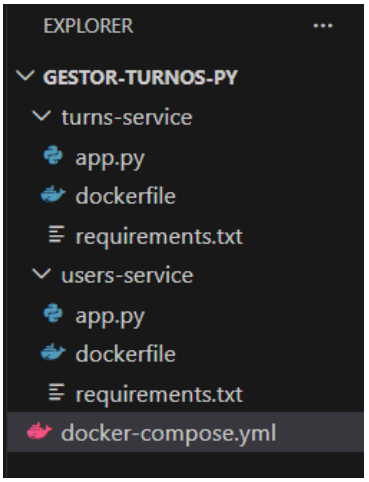
La comunicación entre los diferentes servicios del sistema se realizará mediante peticiones HTTP utilizando una arquitectura basada en API REST. Este tipo de comunicación permite que los servicios intercambien información de manera sencilla y organizada, facilitando la interacción entre los diferentes componentes del sistema distribuido.

2. PROTOTIPO TÉCNICO INICIAL

En esta sección se presenta el prototipo técnico inicial del sistema, donde se muestran las primeras pruebas de funcionamiento de la ejecución de los contenedores en Docker, lo cual permitió verificar que la infraestructura del sistema funciona correctamente antes de continuar con el desarrollo de las demás funcionalidades.

- Orden de las carpetas:

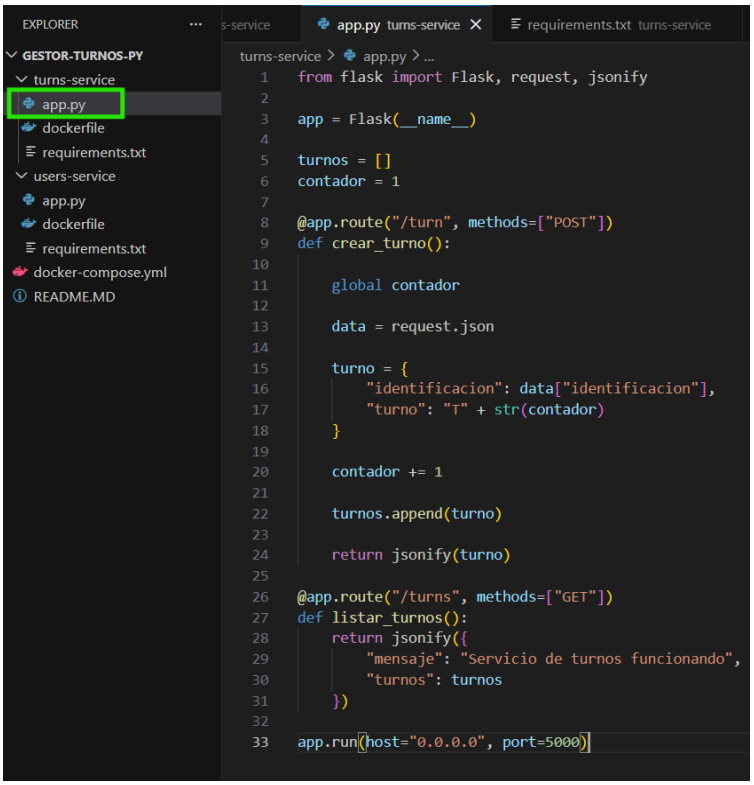
  GESTOR-TURNOS-PY
  docker-compose.yml
  turns-service
  app.py
  Dockerfile
  requirements.txt
  users-service
  app.py
  Dockerfile
  requirements.txt



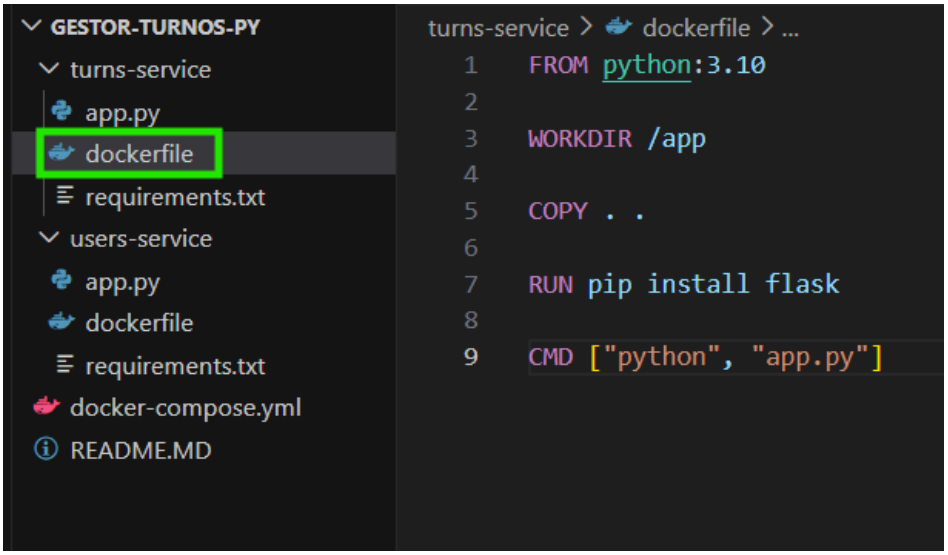
- Códigos de los archivos

Para el servicio de turnos:

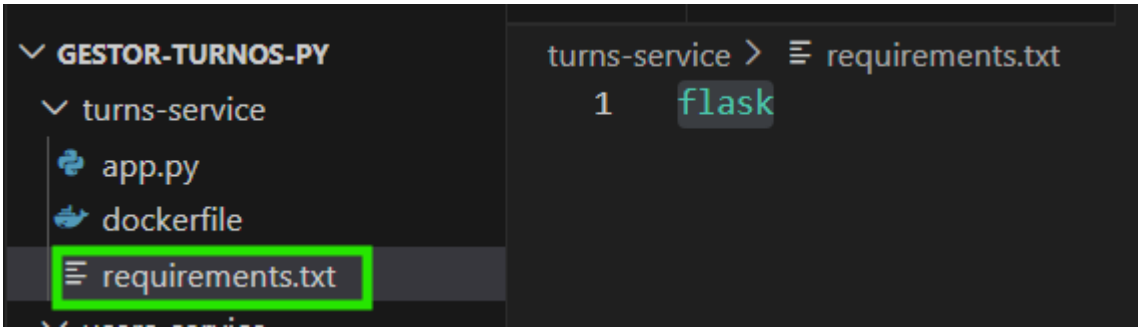
- app.py de turnos



- Dockerfile de turnos:



- Requirements de turnos:



Para el servicio de usuarios:

- app.py de usuarios:

```
GESTOR-TURNOS-PY
turns-service
app.py
dockerfile
requirements.txt
users-service
app.py
dockerfile
requirements.txt
docker-compose.yml
README.MD
```

users-service > app.py > ...

```python
from flask import Flask, request, jsonify

app = Flask(__name__)

users = []

@app.route("/users", methods=["GET"])
def get_users():
    return jsonify({
        "mensaje": "Servicio de usuarios funcionando",
        "usuarios_registrados": users
    })

@app.route("/users", methods=["POST"])
def create_user():
    data = request.json

    user = {
        "identificacion": data["identificacion"],
        "telefono": data["telefono"]
    }

    users.append(user)

    return jsonify({"mensaje": "usuario creado", "user": user})

app.run(host="0.0.0.0", port=5000)
```

- Dockerfile de usuarios:

```
GESTOR-TU...
turns-service
app.py
dockerfile
requirements.txt
users-service
app.py
dockerfile
requirements.txt
docker-compose.yml
README.MD
```

users-service > dockerfile > ...

```dockerfile
FROM python:3.10

WORKDIR /app

COPY . .

RUN pip install flask

CMD ["python", "app.py"]
```

- Requirements de usuarios:

```
GESTOR-TURNOS-PY
turns-service
app.py
dockerfile
requirements.txt
users-service
app.py
dockerfile
requirements.txt
docker-compose.yml
```

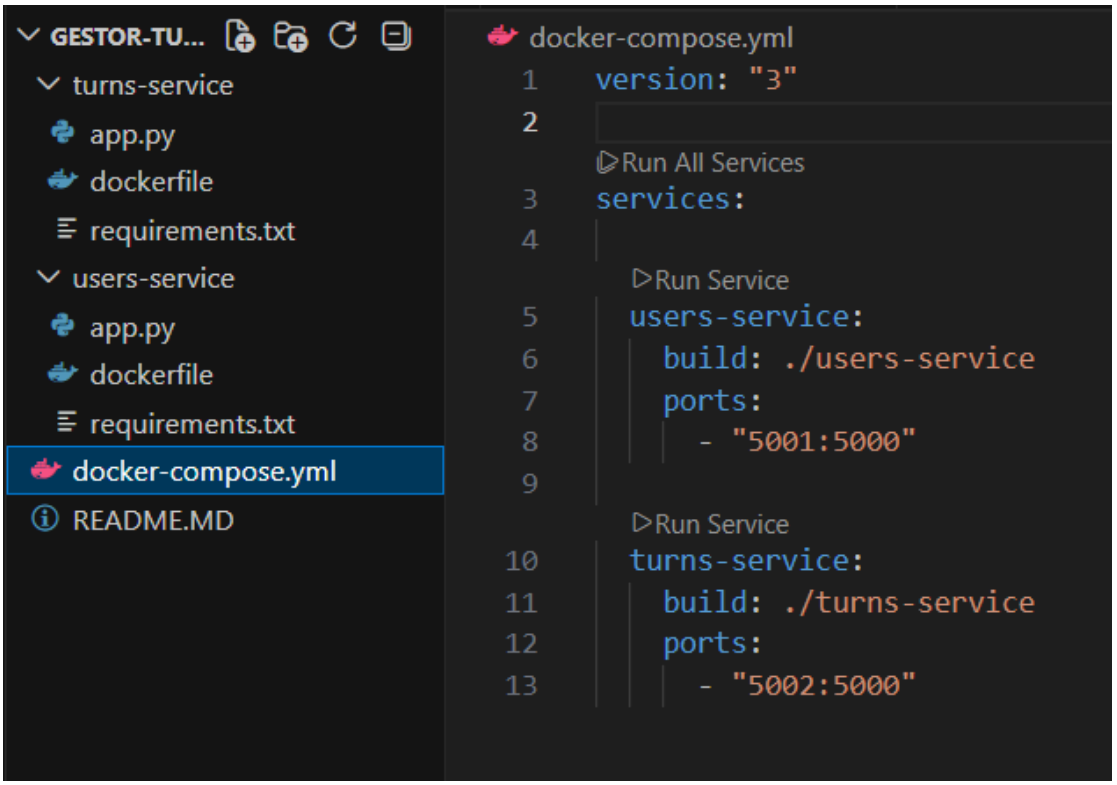
users-service > requirements.txt

```
flask
```

Docker compose funcionando

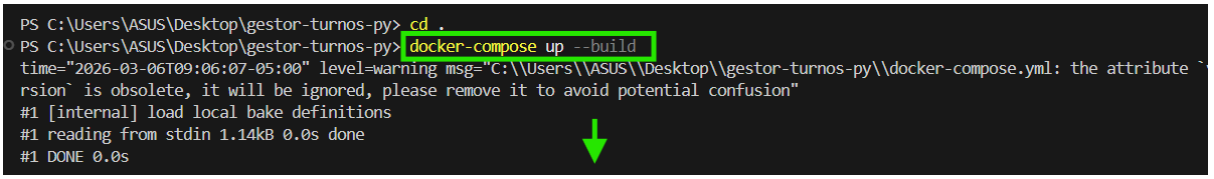
El archivo **docker-compose.yml** es el encargado de definir y coordinar los diferentes servicios que conforman el sistema dentro de contenedores. En el avance del proyecto se utiliza para configurar y ejecutar los microservicios de usuarios y turnos, para especificar los puertos de acceso. De esta manera, Docker Compose permite iniciar todo el sistema con un solo comando, facilitando la ejecución, organización y comunicación entre los servicios que componen la arquitectura distribuida.

- Docker-compose.yml:



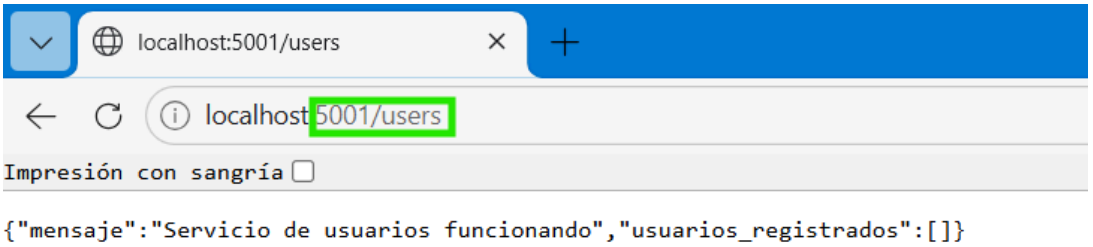
- Ejecutar el sistema desde la terminal con docker-compose up –build

  Con este comando Docker va a crear y levantar los servicios, es necesario realizar este paso cada vez que se quiera utilizar.

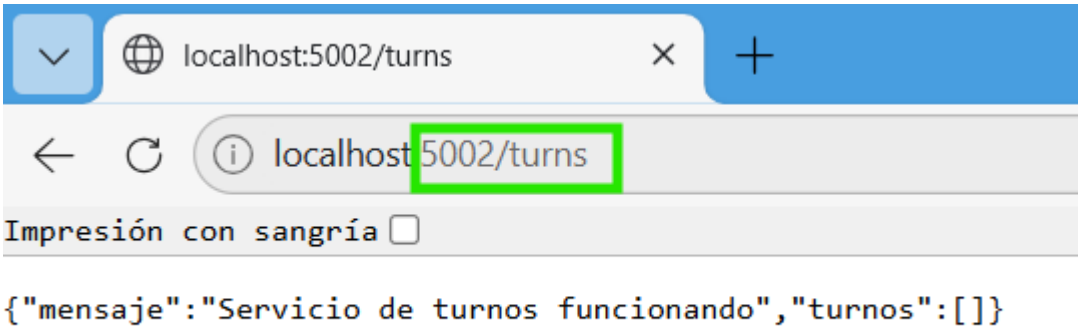
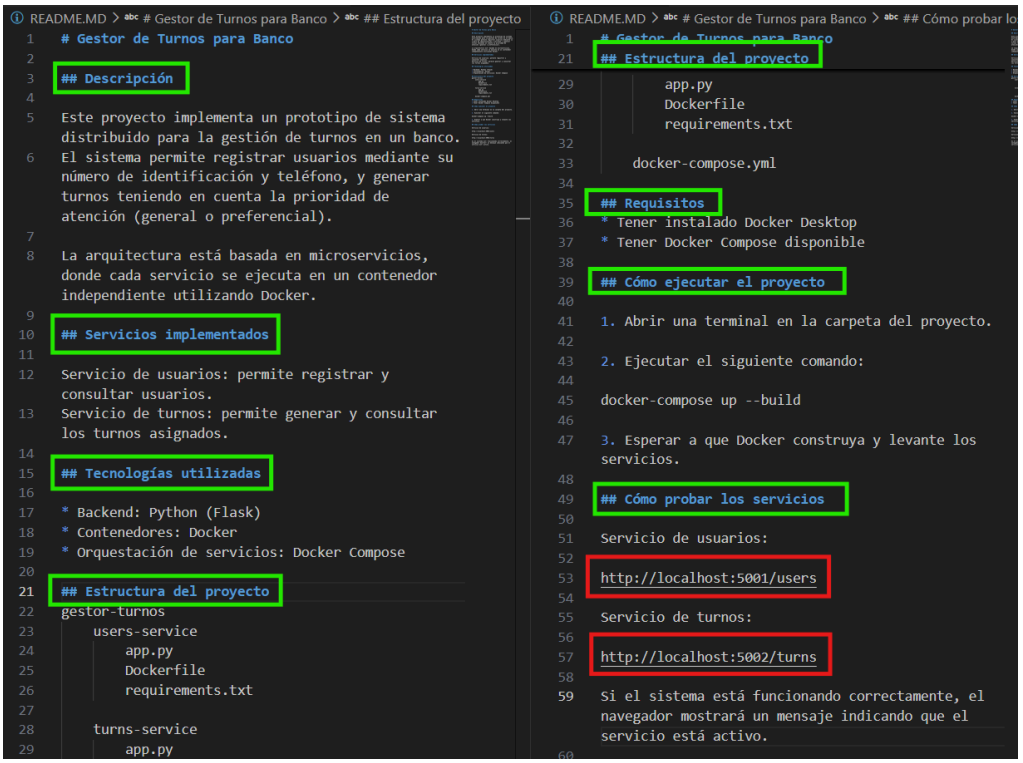


- Para probar si funciona, en el navegador debemos buscar localhost y el puerto correspondiente.

Para servicio de usuarios: puerto 5001:



Para servicio de turnos: puerto 5002:



README